Лабораторна робота №6. Progressive delivery: canary з Argo Rollouts

Дисципліна: «Технології DevOps» · Студент: MarcoGr

Репозиторій: https://github.com/MarcoGr11/DevOps-gitops

Середовище: Kubernetes v1.37.0 (minikube, kubectl v1.37.1), Argo CD v3.5.6, Argo Rollouts v1.10.1 (чарт argo-rollouts 2.43.8, плагін kubectl v1.10.0), Helm v3.22.0

Проєкт із лаби №5 (система бронювання коворкінгу під керуванням Argo CD) доповнено поступовими релізами для api. Контролер Argo Rollouts встановлюється з Git окремим Application `argo-rollouts`. У чарті `charts/booking` прапорець `rollout.enabled` замінює Deployment `booking-api` на Rollout із тим самим шаблоном пода; worker і postgres лишаються Deployment. У `values-minikube.yaml` 5 реплік і стратегія canary: `setWeight` 20, 60, 80, ручна пауза `pause: {}` після 20% і паузи по 30 с після 60% і 80%. Маршрутизатора трафіку немає, тому вага задається часткою подів за звичайним Service. Canary обрано тому, що api не зберігає стан між запитами, нову версію можна перевірити на малій частці запитів, а додаткових подів і preview-сервісу, як у blue-green, не потрібно. Образи `ghcr.io/marcogr11/devops-api` мають теги 1.1.0, 1.2.0 і 1.3.0, версію видно в `/api/health/`.

1. Контролер Argo Rollouts через Argo CD

1. Контролер і плагін встановлено

Перевіряємо, що Application `argo-rollouts` з Git розгорнув контролер і CRD, і що встановлено плагін `kubectl argo rollouts`.

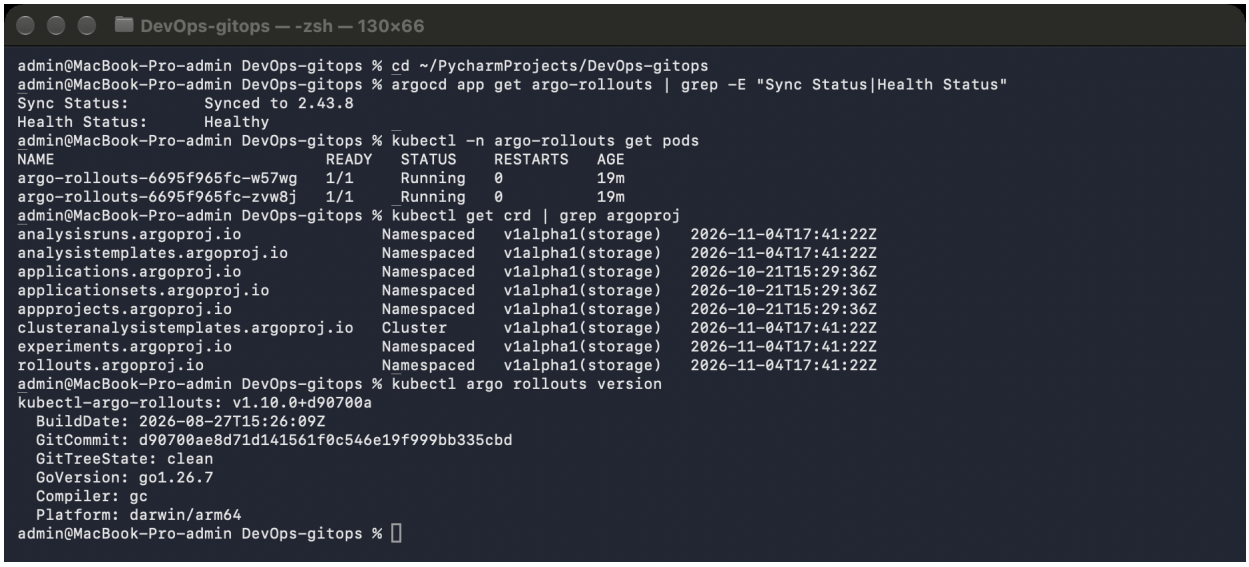


Рис. 1. Контролер Argo Rollouts, його CRD і плагін kubectl

Application `argo-rollouts` має стан `Synced to 2.43.8` і `Healthy`. У namespace `argo-rollouts` працюють 2 поди контролера (`1/1`, `Running`, 0 перезапусків). Серед CRD групи `argoproj.io` є `rollouts`, `analysistemplates`, `analysisruns`, `clusteranalysistemplates` і `experiments`, створені 2026-11-04, тобто разом із контролером; `applications`, `applicationsets` і `appprojects` лишилися від Argo CD (2026-10-21). Плагін: `kubectl-argo-rollouts: v1.10.0+d90700a`, `darwin/arm64`.

2. Rollout у Helm-чарті

2. Шаблон Rollout замість Deployment

Показуємо шаблони чарту і те, що з `values-minikube.yaml` рендериться Rollout, а не Deployment для api.

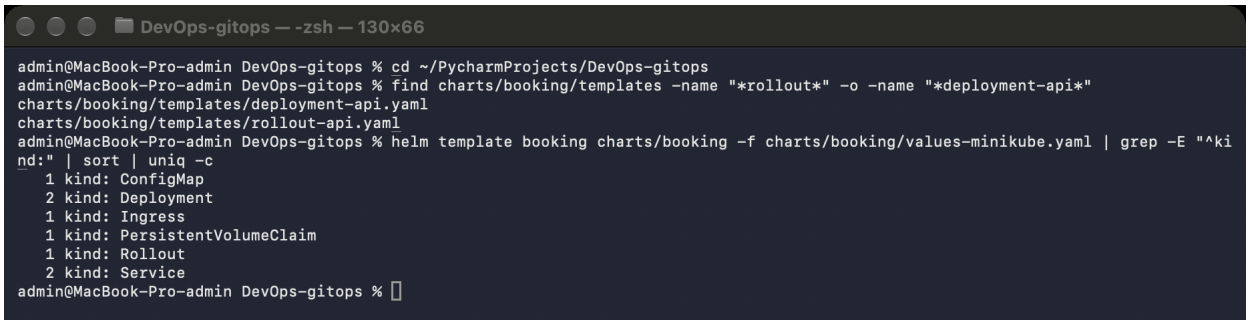


Рис. 2. Шаблони Rollout і Deployment, перелік об'єктів після рендеру

У шаблонах чарту є і `deployment-api.yaml`, і `rollout-api.yaml`. З `values-minikube.yaml` рендериться `1 kind: Rollout` і `2 kind: Deployment` (worker і postgres), тобто Deployment для api замінено. Також є ConfigMap, Ingress, PersistentVolumeClaim і 2 Service; Secret, Job і Namespace у рендері відсутні.

3. Стратегія розгортання

3. Canary у values і Rollout на базовій версії

Показуємо кількість реплік і кроки canary у values, потім Rollout на стабільній версії.

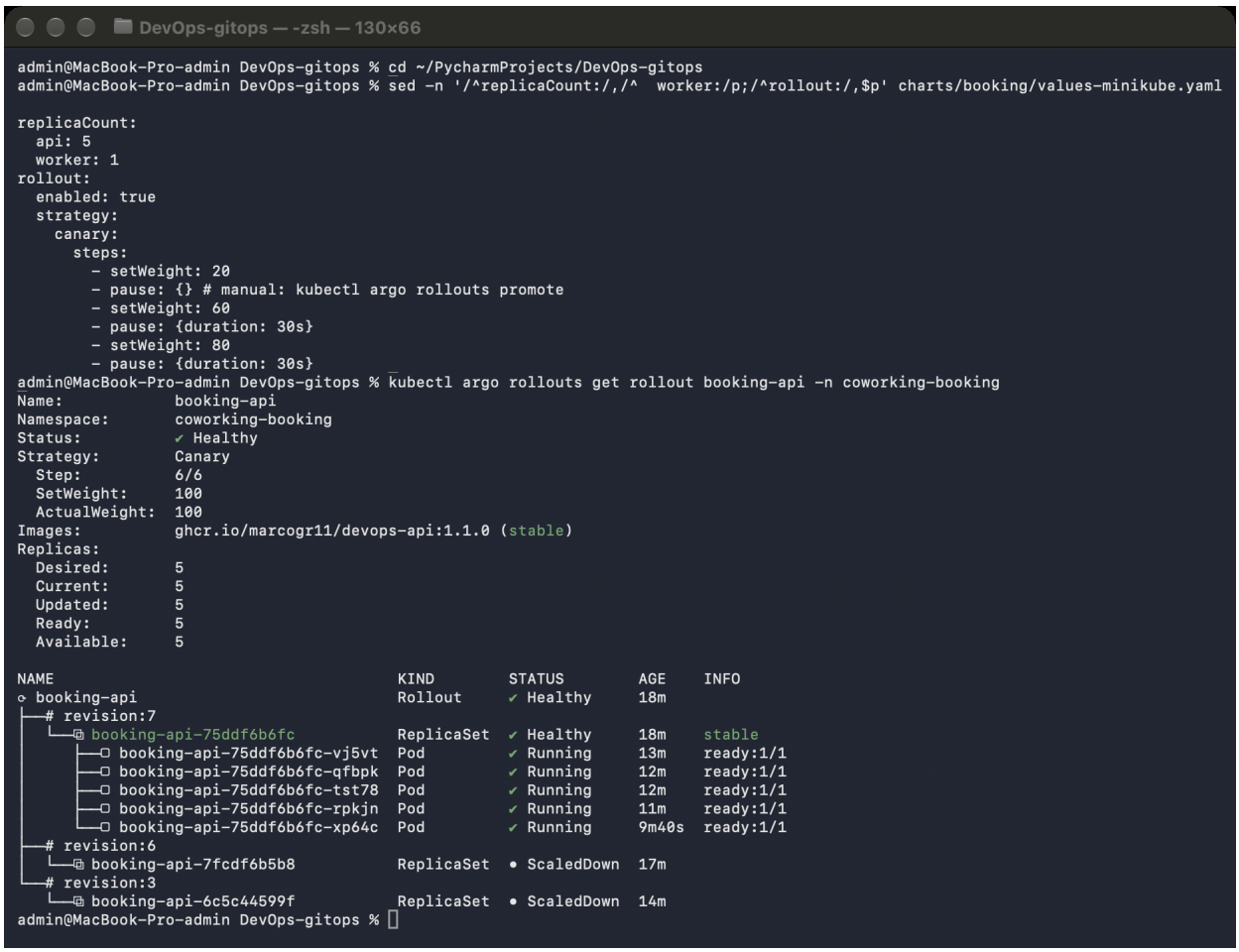


Рис. 3. Кроки canary у values-minikube.yaml і Rollout у стані Healthy на версії 1.1.0

У values задано 5 реплік api (`replicaCount.api: 5`) і `rollout.enabled: true`. Кроки canary: `setWeight` 20, ручна пауза `pause: {}`, `setWeight` 60, пауза 30 с, `setWeight` 80, пауза 30 с. Rollout перебуває у стані `Healthy`, `Step: 6/6`, `SetWeight: 100`, образ `devops-api:1.1.0 (stable)`, усі 5 подів `Running` і `ready:1/1` в одному ReplicaSet.

4. Реліз через Git

4.а Реліз комітом

Змінюємо тег образу api з 1.1.0 на 1.2.0, комітимо і пушимо.

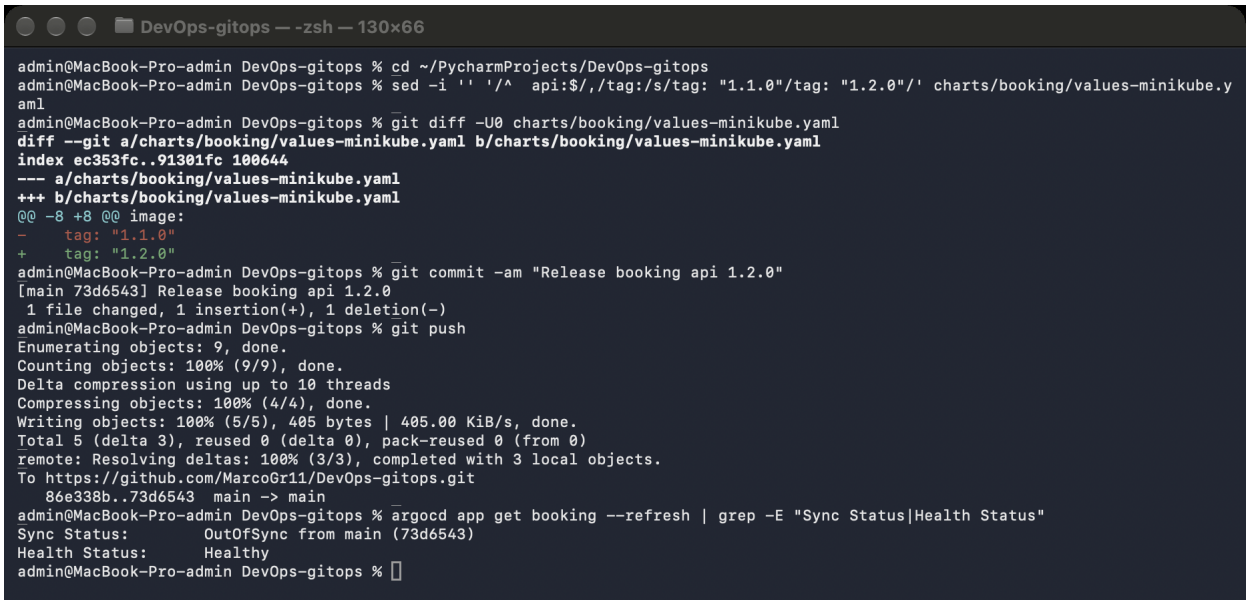


Рис. 4. Реліз 1.2.0: зміна тегу, коміт і push у main

`git diff` показує зміну одного рядка (`tag: "1.1.0"` на `tag: "1.2.0"`, рядок 8, тег api); тег worker не змінено. Коміт `73d6543 Release booking api 1.2.0` запушено в `main`. Після refresh Argo CD показує `OutOfSync from main (73d6543)` і ще `Healthy`: розбіжність виявлено, реліз іще не почався.

4.б Пауза на 20%

Чекаємо на першу ручну паузу і показуємо стан релізу в CLI та Argo CD.

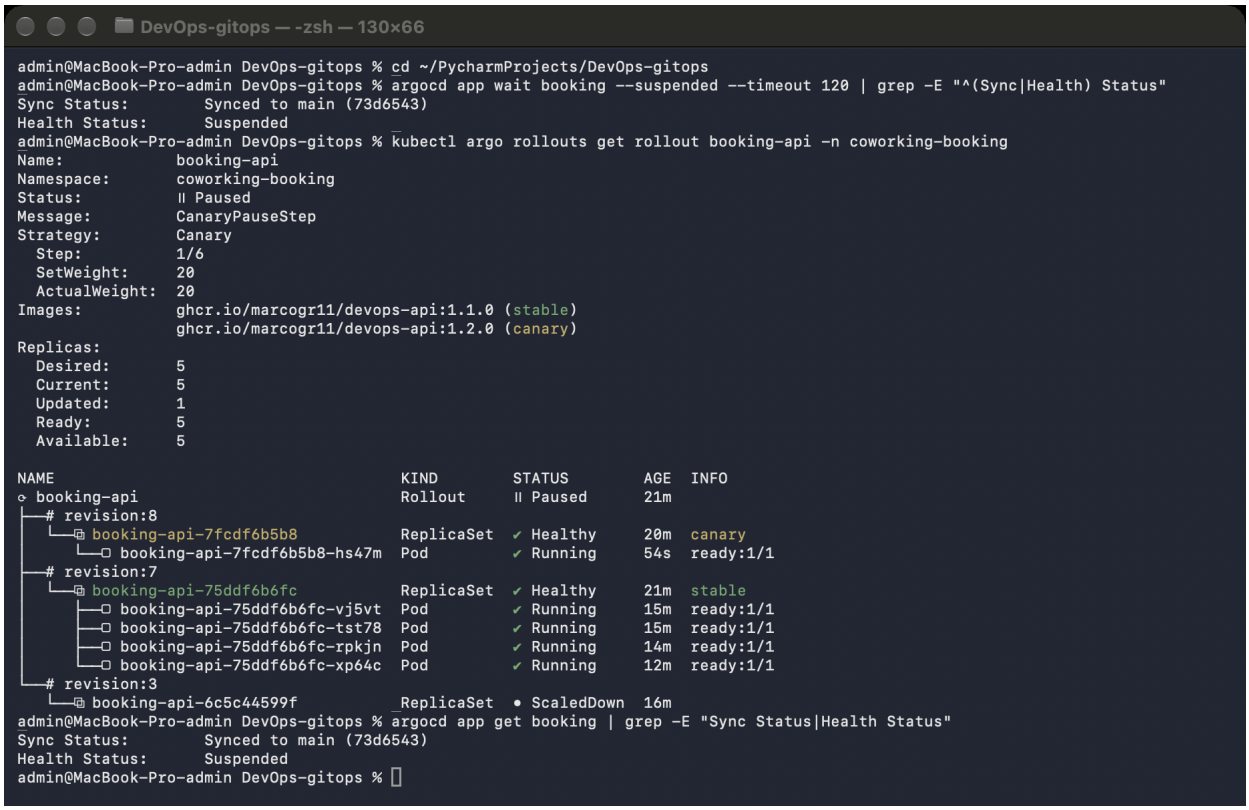


Рис. 5. Реліз на ручній паузі: 1 под canary і 4 поди stable, застосунок Suspended

Застосунок `Synced to main (73d6543)`, `Health Status: Suspended`. Rollout у стані `Paused`, `Message: CanaryPauseStep`, `Step: 1/6`, `SetWeight: 20`, `ActualWeight: 20`. Працюють два образи: `1.1.0 (stable)` і `1.2.0 (canary)`. У дереві 1 под canary (revision:8) і 4 поди stable (revision:7), `Updated: 1`, `Ready: 5`: кількість подів не зменшилась.

4.в Відповіді обох версій

Надсилаємо 40 запитів із пода в кластері через Service і рахуємо відповіді кожної версії.

```
DevOps-gitops — -zsh — 130×66

admin@MacBook-Pro-admin DevOps-gitops % cd ~/PycharmProjects/DevOps-gitops
admin@MacBook-Pro-admin DevOps-gitops % kubectl -n coworking-booking run curl --rm -i --restart=Never --image=curlimages/curl -- s
h -c 'for i in $(seq 1 40); do curl -s http://booking-api/api/health/; echo; sleep 0.3; done' | grep -o '"version":"[^"]*"' | sort
_| uniq -c
All commands and output from this session will be recorded in container logs, including credentials and sensitive information pass
ed through the command prompt.
If you don't see a command prompt, try pressing enter.
  31 "version":"1.1.0"
   9 "version":"1.2.0"
admin@MacBook-Pro-admin DevOps-gitops %
```

Рис. 6. Цикл із 40 запитів усередині кластера: відповідають обидві версії

Із 40 запитів через Service `booking-api` 31 обслужила версія 1.1.0, а 9 — версія 1.2.0, тобто 22,5% при вазі 20%. Частка наближена, бо Service розподіляє запити між подами випадково. Запити йшли з пода в кластері, бо port-forward прив'язаний до одного пода й обох версій не показав би. Рядки про запис сесії виводить `kubectl run`.

4.г Інтерфейс Argo CD на паузі

Відкриваємо https://localhost:8080 → `booking` і показуємо дерево з Rollout на паузі.

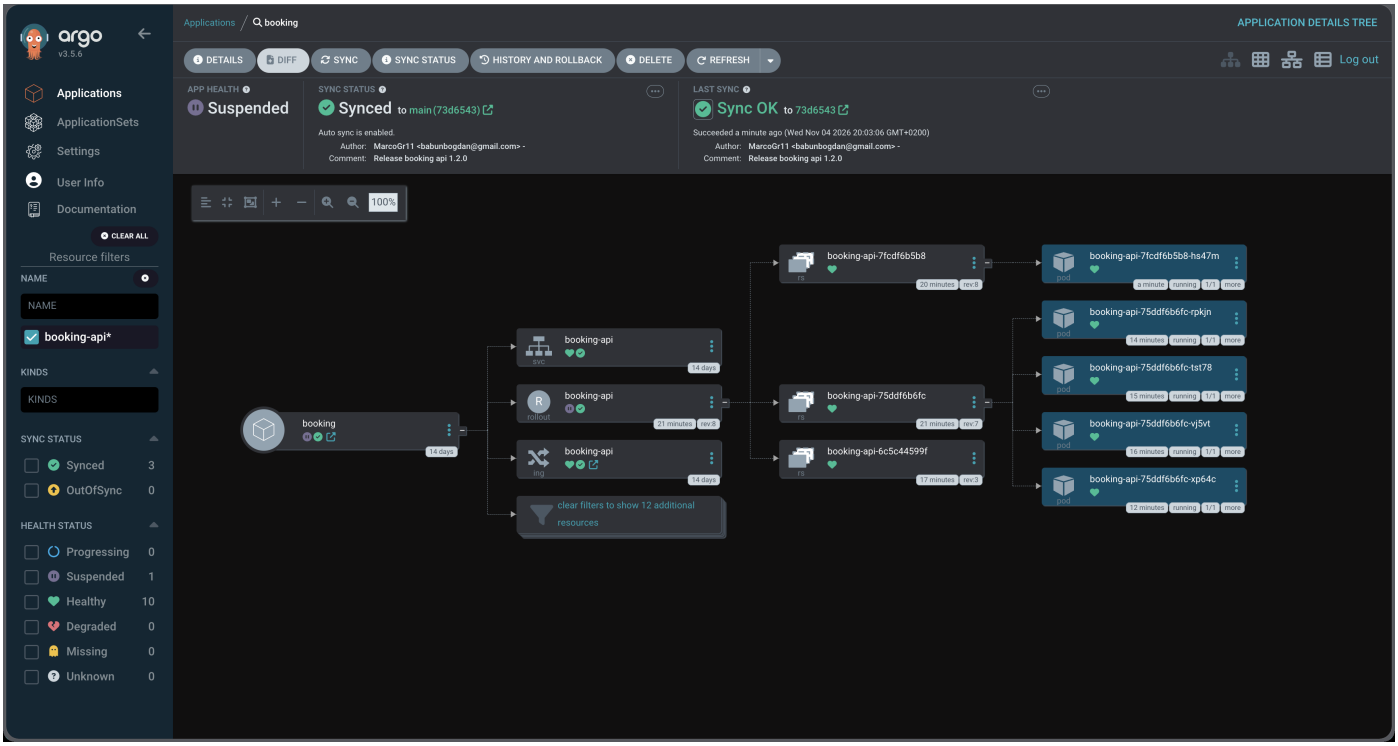


Рис. 7. Дерево застосунку booking в Argo CD: Rollout на ручній паузі

В інтерфейсі Argo CD застосунок `Suspended`, `Synced to main (73d6543)`, останній sync — коміт «Release booking api 1.2.0». Дерево відфільтровано за іменем `booking-api*`, тому видно лише об'єкти api, а решту 12 об'єктів приховано фільтром. Rollout `booking-api` (rev:8) має значок паузи. Під ним ReplicaSet `7fcdf6b5b8` (rev:8) з 1 подом, `75ddf6b6fc` (rev:7) з 4 подами і `6c5c44599f` (rev:3) без подів. Панель браузера обрізано.

4.д Promote і завершення релізу

Виконуємо promote на ручній паузі, чекаємо проходження 60%, 80%, 100% і повторюємо цикл запитів.

```
DevOps-gitops — -zsh — 130×66

admin@MacBook-Pro-admin DevOps-gitops % cd ~/PycharmProjects/DevOps-gitops
admin@MacBook-Pro-admin DevOps-gitops % kubectl argo rollouts promote booking-api -n coworking-booking
rollout 'booking-api' promoted
admin@MacBook-Pro-admin DevOps-gitops % kubectl argo rollouts status booking-api -n coworking-booking --timeout 300s
Progressing - more replicas need to be updated
Paused - CanaryPauseStep
Progressing - more replicas need to be updated
Paused - CanaryPauseStep
Progressing - more replicas need to be updated
Progressing - updated replicas are still becoming available
Healthy
admin@MacBook-Pro-admin DevOps-gitops % kubectl -n coworking-booking run curl --rm -i --restart=Never --image=curlimages/curl -- s
h -c 'for i in $(seq 1 40); do curl -s http://booking-api/api/health/; echo; sleep 0.3; done' | grep -o '"version":"[^"]*"' | sort
_| uniq -c
All commands and output from this session will be recorded in container logs, including credentials and sensitive information pass
ed through the command prompt.
If you don't see a command prompt, try pressing enter.
   40 "version":"1.2.0"
admin@MacBook-Pro-admin DevOps-gitops %
```

Рис. 8. Promote: реліз дійшов до Healthy, усі відповіді від 1.2.0

Після `promote` команда `status` показує проходження кроків: двічі `Paused - CanaryPauseStep` (таймерні паузи на 60% і 80%) і в кінці `Healthy`. Вручну нічого більше не підтверджувалось. Повторний цикл запитів дає `40 "version":"1.2.0"`: стара версія більше не відповідає.

5. Скасування релізу

5.а Реліз 1.3.0 і abort

Запускаємо ще один реліз, чекаємо паузи і скасовуємо його на проміжному кроці.

```
DevOps-gitops — -zsh — 130×66

admin@MacBook-Pro-admin DevOps-gitops % cd ~/PycharmProjects/DevOps-gitops
admin@MacBook-Pro-admin DevOps-gitops % sed -i '' '/^  api:$/,/tag:/s/tag: "1.2.0"/tag: "1.3.0"/' charts/booking/values-minikube.y
aml
admin@MacBook-Pro-admin DevOps-gitops % git commit -am "Release booking api 1.3.0"
[main 7a54473] Release booking api 1.3.0
 1 file changed, 1 insertion(+), 1 deletion(-)
admin@MacBook-Pro-admin DevOps-gitops % git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 405 bytes | 405.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/MarcoGr11/DevOps-gitops.git
   73d6543..7a54473  main -> main
admin@MacBook-Pro-admin DevOps-gitops % argocd app get booking --refresh | grep -E "Sync Status|Health Status"
Sync Status:        OutOfSync from main (7a54473)
Health Status:      Healthy
admin@MacBook-Pro-admin DevOps-gitops % argocd app wait booking --suspended --timeout 120 | grep -E "^(Sync|Health) Status"
Sync Status:        Synced to main (7a54473)
Health Status:      Suspended
admin@MacBook-Pro-admin DevOps-gitops % kubectl argo rollouts abort booking-api -n coworking-booking
rollout 'booking-api' aborted
admin@MacBook-Pro-admin DevOps-gitops % kubectl argo rollouts get rollout booking-api -n coworking-booking
Name:            booking-api
Namespace:       coworking-booking
Status:          ✖ Degraded
Message:         RolloutAborted: Rollout aborted update to revision 9
Strategy:        Canary
  Step:          0/6
  SetWeight:     0
  ActualWeight:  0
Images:          ghcr.io/marcogr11/devops-api:1.2.0 (stable)
Replicas:
  Desired:       5
  Current:       5
  Updated:       0
  Ready:         4
  Available:     4

NAME                                KIND        STATUS         AGE   INFO
⟳ booking-api                       Rollout     ✖ Degraded     23m
├──# revision:9
│  └──⧉ booking-api-6c5c44599f      ReplicaSet  • ScaledDown   19m   canary
│     └──□ booking-api-6c5c44599f-qnk2l  Pod     ◌ Terminating  8s    ready:1/1
├──# revision:8
│  └──⧉ booking-api-7fcdf6b5b8      ReplicaSet  ◌ Progressing  22m   stable
│     ├──□ booking-api-7fcdf6b5b8-hs47m  Pod     ✔ Running      3m7s  ready:1/1
│     ├──□ booking-api-7fcdf6b5b8-hfn26  Pod     ✔ Running      110s  ready:1/1
│     ├──□ booking-api-7fcdf6b5b8-m8kb7  Pod     ✔ Running      110s  ready:1/1
│     ├──□ booking-api-7fcdf6b5b8-n2mfd  Pod     ✔ Running      74s   ready:1/1
│     ├──□ booking-api-7fcdf6b5b8-zxv5t  Pod     ◌ Terminating  39s   ready:1/1
│     └──□ booking-api-7fcdf6b5b8-zzg9n  Pod     ✔ Running      1s    ready:0/1
└──# revision:7
   └──⧉ booking-api-75ddf6b6fc      ReplicaSet  • ScaledDown   23m
admin@MacBook-Pro-admin DevOps-gitops %
```

Рис. 9. Реліз 1.3.0 скасовано на паузі: Rollout Degraded, stable лишається 1.2.0

Коміт `7a54473 Release booking api 1.3.0` запушено, застосунок дійшов до `Suspended`, і на цій паузі виконано `abort`. Rollout одразу став `Degraded` з повідомленням `RolloutAborted: Rollout aborted update to revision 9, Step: 0/6, SetWeight: 0`, єдиний образ `1.2.0 (stable), Updated: 0`. Кадр знято в момент повернення: canary-под `Terminating`, stable ReplicaSet `Progressing`, новий stable-под `zzg9n` ще `ready:0/1`, тому `Ready: 4`.

5.б Synced + Degraded

Показуємо стан застосунку в Argo CD і те, що користувачі отримують стару версію.

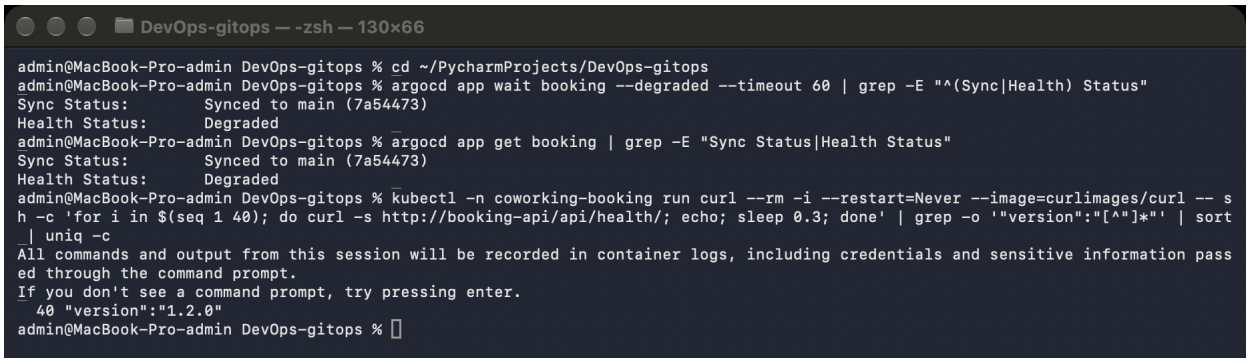


Рис. 10. Synced + Degraded: у Git тег 1.3.0, працює 1.2.0

Застосунок має стан `Synced to main (7a54473)` і `Degraded`, а всі 40 відповідей надходять від 1.2.0. Synced стосується маніфестів: у Git тег 1.3.0, і в Rollout у кластері теж 1.3.0, тож розбіжності немає. Degraded стосується роботи: реліз скасовано, бажаний стан із Git не досягнуто, а поди працюють на попередній версії. Користувачі отримують робочу версію, але Git слід виправити.

5.в Інтерфейс Argo CD у Degraded

Відкриваємо сторінку `booking` в Argo CD і показуємо Degraded.

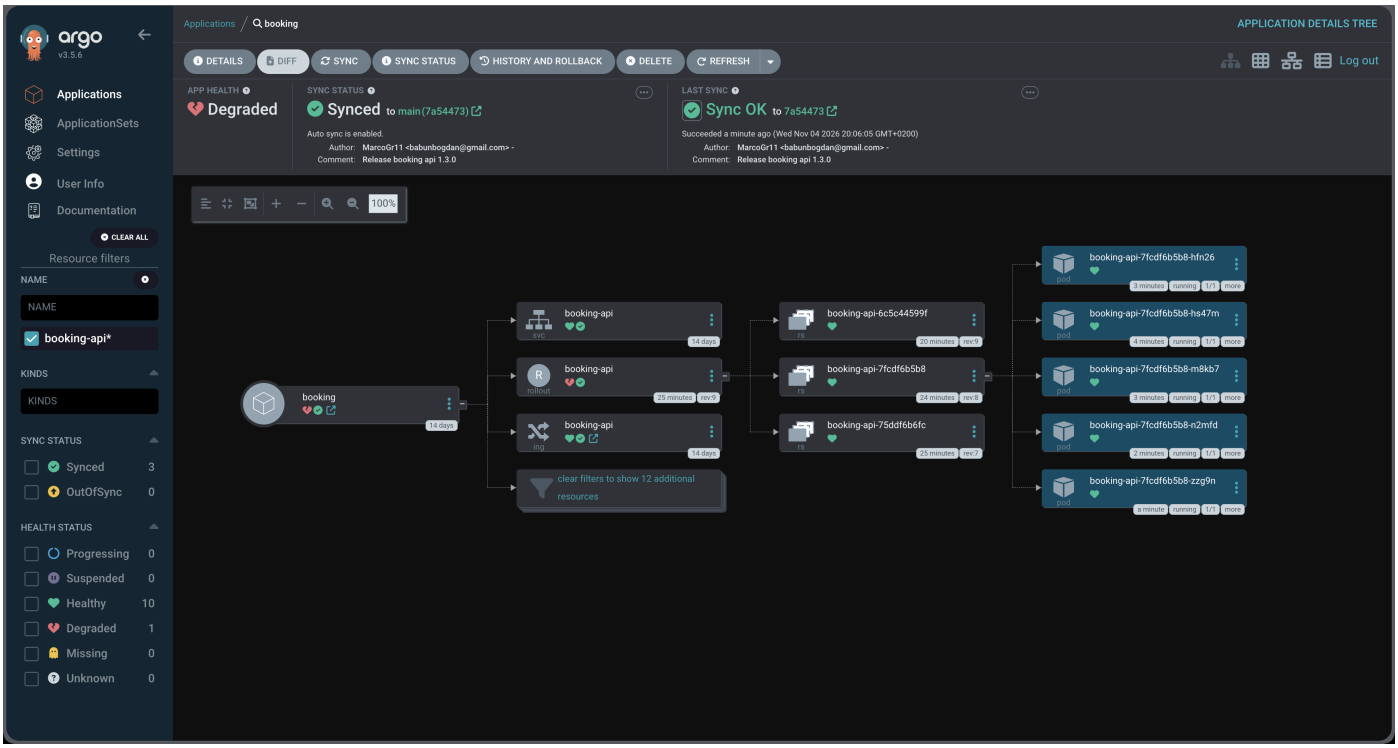


Рис. 11. Застосунок booking в інтерфейсі Argo CD: Synced і Degraded

В інтерфейсі застосунок `Degraded`, `Synced to main (7a54473)`, останній sync — коміт «Release booking api 1.3.0». Rollout `booking-api` (rev:9) позначено червоним значком Degraded, а в лічильнику Health Status є 1 Degraded об'єкт. Canary ReplicaSet `6c5c44599f` (rev:9) уже без подів, 5 подів працюють у `7fcdf6b5b8` (rev:8, версія 1.2.0). Дерево відфільтровано за іменем `booking-api*`, панель браузера обрізано.

5.г Завершення відкату через Git

Скасовуємо коміт із 1.3.0 через `git revert` і перевіряємо, що застосунок знову здоровий.

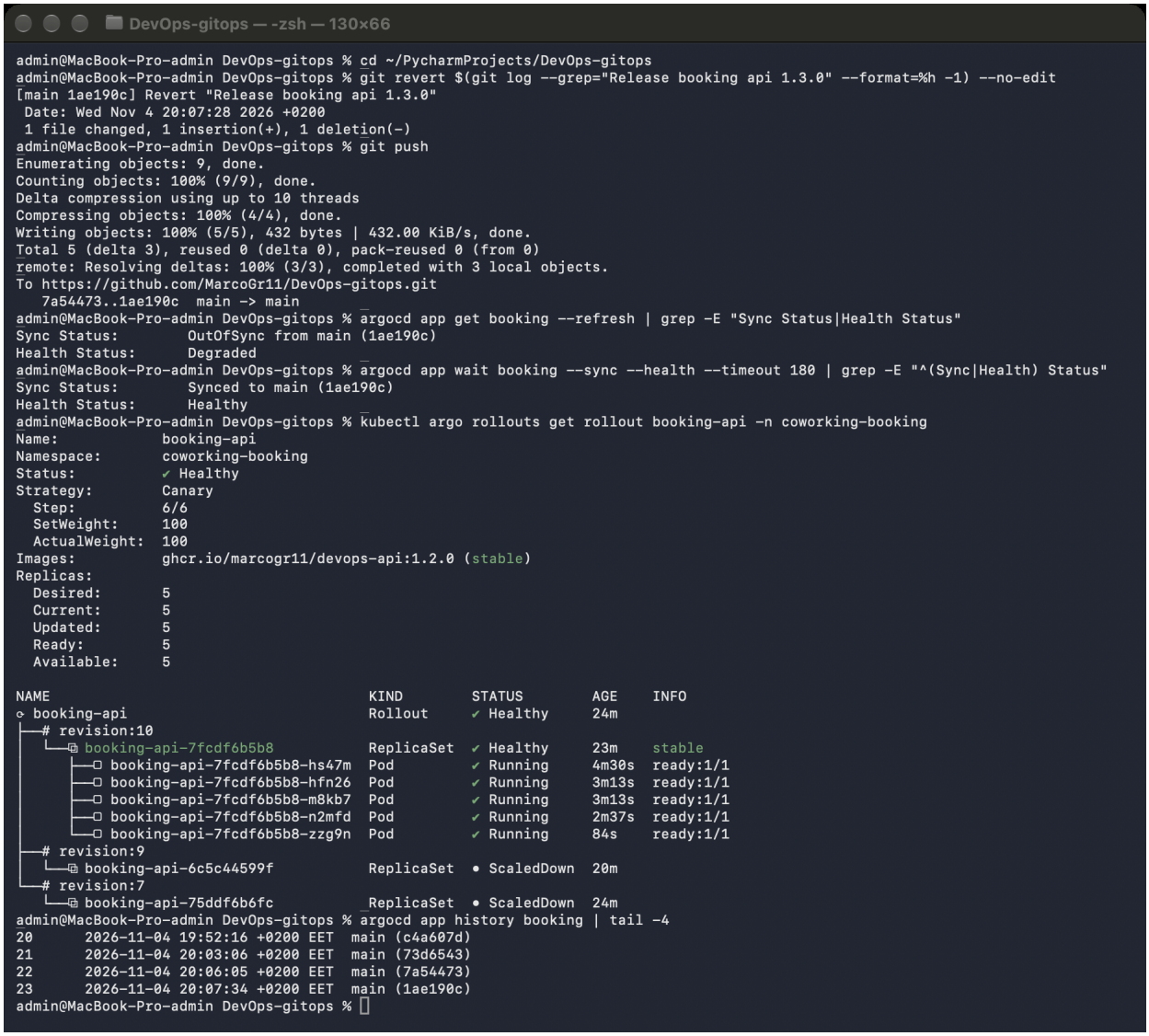


Рис. 12. Відкат через git revert: Healthy на версії 1.2.0, історія ревізій

`git revert` створив коміт `1ae190c Revert "Release booking api 1.3.0"`, і його запушено в `main`. Після refresh стан `OutOfSync`, `Degraded`, далі `Synced to main (1ae190c)`, `Healthy`. Rollout: `Healthy`, `Step: 6/6`, `1.2.0 (stable)`, 5 подів. Поди мають ті самі імена, що й після abort (`hs47m`, `hfn26`, `m8kb7`, `n2mfd`, `zzg9n`): кроків canary не було, бо тег у Git знову збігся зі stable, змінився лише номер ревізії на 10. В історії Argo CD останні ревізії — `73d6543`, `7a54473` і `1ae190c`.

Висновок

Під час rolling update в лабах №3 і №5 Deployment замінює поди безперервно: зупинитись на проміжній частці й перевірити нову версію можна лише вручну, а скасування означає ще один реліз назад. Canary з Argo Rollouts ділить реліз на кроки: нова версія спершу отримує приблизно 20% запитів і чекає ручного рішення promote. Команда abort одразу повертає всі поди на stable-версію, тому користувачі майже не зачеплені невдалим релізом. Git і далі лишається джерелом правди: після abort застосунок має стан Synced і Degraded, доки відкат не завершено через git revert.